

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5 , BL6)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 50

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Code of Conduct:

*I Gurusurya G 192311332 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Gurusurya
Signature of the student:

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

1. Write pseudocode to simulate a simplified TLS handshake process, including client request, server authentication using a digital certificate, session key generation, and secure data exchange.
2. Develop pseudocode for a firewall rule engine that filters incoming packets based on IP address, port number, and protocol, and decides whether to allow or block traffic.
3. Write pseudocode to detect and classify malware behavior (virus, worm, or Trojan) based on propagation method, file modification patterns, and user interaction.
4. Design pseudocode for an Intrusion Detection System (IDS) that uses both signature-based and anomaly-based detection to identify suspicious network activity.
5. Create pseudocode for an Intrusion Prevention System (IPS) that monitors network traffic in real time and automatically blocks malicious requests while logging intrusion attempts.

1. Simplified TLS Handshake Simulation

The pseudocode below models a simplified TLS handshake between a client and a server: the client sends a hello/request, the server proves its identity with a digital certificate, both sides derive a shared session key, and encrypted application data is then exchanged.

```
BEGIN TLS_Handshake_Simulation

// ----- Step 1: Client Hello / Request -----
FUNCTION ClientHello():
    generate client_random_number
    supported_cipher_suites = ["AES256-GCM", "ChaCha20"]
    SEND message(type = "CLIENT_HELLO",
                  client_random,
                  supported_cipher_suites) TO Server
    RETURN client_random

// ----- Step 2: Server Authentication -----
FUNCTION ServerHello_And_Certificate(client_random):
    generate server_random_number
    chosen_cipher = SELECT_CIPHER(supported_cipher_suites)
    certificate = LOAD_SERVER_CERTIFICATE()
    SEND message(type = "SERVER_HELLO",
                  server_random,
                  chosen_cipher,
                  certificate) TO Client
    RETURN server_random, certificate

FUNCTION ClientVerifiesCertificate(certificate):
    trusted_CA_list = LOAD_TRUSTED_ROOT_CAs()
    IF certificate.issuer NOT IN trusted_CA_list:
        ABORT_HANDSHAKE("Untrusted certificate authority")
    IF CURRENT_DATE() > certificate.expiry_date:
        ABORT_HANDSHAKE("Certificate expired")
    IF NOT VERIFY_SIGNATURE(certificate, issuer_public_key):
        ABORT_HANDSHAKE("Invalid certificate signature")
    RETURN certificate.public_key // server's authenticated public key

// ----- Step 3: Session Key Generation -----
FUNCTION GenerateSessionKey(client_random, server_random, server_public_key):
    pre_master_secret = GENERATE_RANDOM_SECRET()
    encrypted_secret = ENCRYPT(pre_master_secret, server_public_key)
    SEND message(type = "KEY_EXCHANGE", encrypted_secret) TO Server

    // Server decrypts using its private key
    pre_master_secret_at_server = DECRYPT(encrypted_secret, server_private_key)

    // Both sides derive the same session key independently
    session_key = KEY_DERIVATION_FUNCTION(client_random,
                                           server_random,
                                           pre_master_secret)

    RETURN session_key

// ----- Step 4: Handshake Confirmation -----
FUNCTION FinishHandshake(session_key):
    client_finished = ENCRYPT("Client handshake complete", session_key)
    SEND client_finished TO Server

    server_finished = ENCRYPT("Server handshake complete", session_key)
    SEND server_finished TO Client

    IF DECRYPT(client_finished, session_key) is valid AND
```

```

        DECRYPT(server_finished, session_key) is valid:
        RETURN "HANDSHAKE_SUCCESS"
    ELSE:
        ABORT_HANDSHAKE("Finished message verification failed")

// ----- Step 5: Secure Data Exchange -----
FUNCTION SecureDataExchange(session_key, plaintext_message):
    ciphertext = ENCRYPT(plaintext_message, session_key)
    SEND ciphertext TO Recipient
    // Recipient side:
    received_plaintext = DECRYPT(ciphertext, session_key)
    RETURN received_plaintext

// ----- Main Driver -----
MAIN:
    client_random = ClientHello()
    (server_random, certificate) = ServerHello_And_Certificate(client_random)
    server_public_key = ClientVerifiesCertificate(certificate)
    session_key = GenerateSessionKey(client_random, server_random,
server_public_key)
    status = FinishHandshake(session_key)

    IF status == "HANDSHAKE_SUCCESS":
        SecureDataExchange(session_key, "Hello, secure world!")
    ELSE:
        TERMINATE_CONNECTION()

END TLS_Handshake_Simulation

```

2. Firewall Rule Engine

This pseudocode implements a simple firewall rule engine. Each incoming packet is checked against an ordered list of rules that match on source/destination IP address, port number, and protocol. The engine returns an ALLOW or BLOCK decision, applying the first matching rule and falling back to a default policy.

```

BEGIN Firewall_Rule_Engine

// Each rule: { action, src_ip, dst_ip, port, protocol, priority }
rule_table = LOAD_FIREWALL_RULES()           // sorted by priority (ascending)
DEFAULT_POLICY = "BLOCK"                     // deny-by-default posture

STRUCT Packet:
    src_ip
    dst_ip
    dst_port
    protocol          // e.g. TCP, UDP, ICMP

FUNCTION MatchesRule(packet, rule):
    IF rule.src_ip != "ANY" AND rule.src_ip != packet.src_ip:
        RETURN FALSE
    IF rule.dst_ip != "ANY" AND rule.dst_ip != packet.dst_ip:
        RETURN FALSE
    IF rule.port != "ANY" AND rule.port != packet.dst_port:
        RETURN FALSE
    IF rule.protocol != "ANY" AND rule.protocol != packet.protocol:
        RETURN FALSE
    RETURN TRUE

FUNCTION EvaluatePacket(packet):
    FOR EACH rule IN rule_table (in priority order):
        IF MatchesRule(packet, rule):
            LOG_EVENT(packet, rule.action, rule)
            RETURN rule.action          // "ALLOW" or "BLOCK"

```

```

// No rule matched
LOG_EVENT(packet, DEFAULT_POLICY, "default-policy")
RETURN DEFAULT_POLICY

FUNCTION ProcessIncomingTraffic():
    WHILE network_interface HAS incoming_packet:
        packet = READ_NEXT_PACKET()
        decision = EvaluatePacket(packet)
        IF decision == "ALLOW":
            FORWARD_PACKET(packet)
        ELSE:
            DROP_PACKET(packet)

MAIN:
    ProcessIncomingTraffic()

END Firewall_Rule_Engine

```

3. Malware Behavior Detection & Classification

This pseudocode outlines a rule-based classifier that inspects observed behavior of a suspicious program (propagation method, file modification patterns, and whether it needs user interaction) and labels it as a virus, worm, Trojan, or benign/unknown.

```

BEGIN Malware_Behavior_Classifier

    STRUCT ObservedBehavior:
        propagation_method      // "self-replicating", "network-spreading", "none",
        "disguised"
        file_modification        // "infects_host_files", "creates_new_files", "none"
        requires_user_action     // TRUE / FALSE (e.g., user must open/run it)
        network_activity         // TRUE / FALSE
        spoofs_legitimate_app    // TRUE / FALSE

    FUNCTION ClassifyMalware(behavior):

        // ---- Rule 1: Virus ----
        // Attaches to and modifies existing files; needs a user to execute
        // the host file to spread; does not typically self-propagate over
        // the network on its own.
        IF behavior.file_modification == "infects_host_files" AND
           behavior.requires_user_action == TRUE AND
           behavior.propagation_method == "self-replicating":
            RETURN "VIRUS"

        // ---- Rule 2: Worm ----
        // Spreads autonomously across a network without requiring the user
        // to open or run anything; typically creates copies of itself
        // rather than infecting existing files.
        IF behavior.propagation_method == "network-spreading" AND
           behavior.requires_user_action == FALSE AND
           behavior.network_activity == TRUE:
            RETURN "WORM"

        // ---- Rule 3: Trojan ----
        // Disguises itself as legitimate software, relies on the user to
        // install/run it, and does not self-replicate.
        IF behavior.spoofs_legitimate_app == TRUE AND
           behavior.requires_user_action == TRUE AND
           behavior.propagation_method == "disguised":
            RETURN "TROJAN"

        // ---- Rule 4: No definitive match ----

```

```

    RETURN "UNKNOWN_OR_BENIGN"

FUNCTION AnalyzeSample(sample):
    behavior = MONITOR_SAMPLE_IN_SANDBOX(sample)    // dynamic analysis
    classification = ClassifyMalware(behavior)
    LOG_CLASSIFICATION(sample.name, classification, behavior)
    RETURN classification

MAIN:
    FOR EACH sample IN quarantine_queue:
        result = AnalyzeSample(sample)
        IF result != "UNKNOWN_OR_BENIGN":
            QUARANTINE(sample)
            ALERT_ADMINISTRATOR(sample, result)

END Malware_Behavior_Classifier

```

4. Intrusion Detection System (IDS)

This pseudocode combines two complementary detection strategies. Signature-based detection compares traffic against a database of known attack patterns, while anomaly-based detection flags traffic that statistically deviates from a learned baseline of normal behavior.

```

BEGIN Intrusion_Detection_System

    signature_database = LOAD_KNOWN_ATTACK_SIGNATURES()
    baseline_profile    = LOAD_NORMAL_TRAFFIC_BASELINE()    // learned over time
    ANOMALY_THRESHOLD = 0.85                                // 0..1 deviation score

    STRUCT TrafficEvent:
        src_ip, dst_ip, port, protocol, payload, timestamp, packet_rate

    // ----- Signature-Based Detection -----
    FUNCTION SignatureMatch(event):
        FOR EACH signature IN signature_database:
            IF CONTAINS_PATTERN(event.payload, signature.pattern) OR
               MATCHES_KNOWN_EXPLOIT(event, signature):
                RETURN TRUE, signature.attack_name
        RETURN FALSE, NULL

    // ----- Anomaly-Based Detection -----
    FUNCTION AnomalyScore(event, baseline_profile):
        feature_vector = EXTRACT_FEATURES(event)            // rate, size, ports used,
etc.                                                         // normalized 0..1
        deviation = COMPUTE_STATISTICAL_DEVIATION(feature_vector, baseline_profile)
        RETURN deviation

    // ----- Combined Detection Pipeline -----
    FUNCTION InspectEvent(event):
        (sig_hit, attack_name) = SignatureMatch(event)
        IF sig_hit == TRUE:
            RETURN ALERT("SIGNATURE_MATCH", attack_name, event, severity = "HIGH")

        anomaly_score = AnomalyScore(event, baseline_profile)
        IF anomaly_score >= ANOMALY_THRESHOLD:
            RETURN ALERT("ANOMALY_DETECTED", anomaly_score, event, severity = "MEDIUM")

        RETURN "NORMAL"

    FUNCTION ALERT(alert_type, detail, event, severity):
        record = { alert_type, detail, event, severity, time = NOW() }
        LOG_ALERT(record)
        NOTIFY_SECURITY_TEAM(record)

```

```

    RETURN record

MAIN:
    WHILE network_sensor HAS traffic:
        event = CAPTURE_NEXT_EVENT()
        result = InspectEvent(event)
        IF result != "NORMAL":
            UPDATE_INCIDENT_DASHBOARD(result)
            baseline_profile = UPDATE_BASELINE(baseline_profile, event) // adaptive
learning
END Intrusion_Detection_System

```

5. Intrusion Prevention System (IPS)

The IPS pseudocode extends detection with real-time enforcement: it inspects live traffic, and when malicious activity is confirmed it automatically blocks the offending source, terminates or drops the connection, and logs the intrusion attempt for later review.

```

BEGIN Intrusion_Prevention_System

    active_block_list = EMPTY_SET()           // IPs currently blocked
    detection_engine = IDS_ENGINE             // reuse signature + anomaly detection
    BLOCK_DURATION    = 3600                  // seconds, temporary auto-block

    STRUCT TrafficEvent:
        src_ip, dst_ip, port, protocol, payload, timestamp

    FUNCTION IsAlreadyBlocked(src_ip):
        IF src_ip IN active_block_list AND NOT EXPIRED(src_ip, BLOCK_DURATION):
            RETURN TRUE
        RETURN FALSE

    FUNCTION BlockSource(src_ip, reason):
        ADD src_ip TO active_block_list WITH timestamp = NOW()
        APPLY_FIREWALL_RULE(action = "BLOCK", src_ip = src_ip)
        LOG_INTRUSION_ATTEMPT(src_ip, reason, NOW())
        NOTIFY_SECURITY_TEAM(src_ip, reason)

    FUNCTION InspectAndEnforce(event):
        IF IsAlreadyBlocked(event.src_ip):
            DROP_PACKET(event)
            RETURN "DROPPED_PRE_BLOCKED"

        detection_result = detection_engine.InspectEvent(event)

        IF detection_result != "NORMAL":
            // Real-time automatic prevention
            DROP_PACKET(event)
            TERMINATE_CONNECTION(event.src_ip, event.dst_ip)
            BlockSource(event.src_ip, detection_result.detail)
            RETURN "BLOCKED_AND_LOGGED"
        ELSE:
            FORWARD_PACKET(event)
            RETURN "ALLOWED"

    FUNCTION PeriodicMaintenance():
        FOR EACH src_ip IN active_block_list:
            IF EXPIRED(src_ip, BLOCK_DURATION):
                REMOVE src_ip FROM active_block_list
                REMOVE_FIREWALL_RULE(src_ip)

MAIN:

```

```
    WHILE network_interface HAS live_traffic:
        event = CAPTURE_NEXT_EVENT()
        outcome = InspectAndEnforce(event)
        RECORD_METRIC(outcome)
        PeriodicMaintenance()           // expire old blocks periodically
END Intrusion_Prevention_System
```